

UNIVERSIDAD NACIONAL DE COLOMBIA

Sede Manizales

Facultad de Ingeniería y Arquitectura

Departamento de Ingeniería Eléctrica, Electrónica y Computación

FEMTO UN

Implementación de un Procesador Softcore

RISC-V FemtoRV32 sobre FPGA

Tang Primer 20K

Diseño, Verificación y Aplicación

Proyecto de Grado

Presentado por:

Laura Daniela Alarcón Castaño
Sebastián Orozco Agudelo

Director:

Juan Bernardo Gómez Mendoza

Manizales, Colombia

2026

Índice

Plataforma Base — SoC RISC-V con UART	4
1. Introducción	4
1.1. Descripción del Proyecto	4
1.2. Objetivos	4
1.2.1. Objetivo General	4
1.2.2. Objetivos Específicos	5
1.3. Justificación	5
2. Arquitectura General del SoC	5
2.1. Diagrama de Bloques	5
3. Flujo de Datos — Máquina de Estados	7
3.1. Bus de Memoria	7
4. Descripción Detallada de Hardware	8
4.1. Módulo top_tang_bram — Nivel Superior	8
4.2. Módulo SOC_bram — Integración del Sistema	9
4.3. Módulo bram_hex — Memoria de Programa y Datos	9
5. Módulo FemtoRV32 — Procesador RISC-V	9
6. Mapa de Memoria	10
7. Comunicación UART	11
8. Implementación en FPGA	11
Proyecto 1 — Verificación del Canal UART (Eco ASCII)	11
9. Contexto y Objetivo	12
10. Diseño del Firmware de Eco	12
11. Hallazgos Durante la Depuración	13
11.1. FIX E1 — Codificación UTF-8 en la terminal	13
11.2. FIX E2 — Punteros UART como variables globales	13
11.3. FIX E3 — Doble procesamiento del mismo byte	13
11.4. FIX E4 — BRAM insuficiente	13

12.Resultados del Eco ASCII	14
Proyecto 2 — Calculadora Básica (Sumadora)	15
13.Contexto	15
14.Diseño del Firmware	15
14.1. Acumulación de Dígitos sin Multiplicación	15
14.2. Impresión sin División	15
14.3. Lazo Principal	15
15.Hallazgos Durante la Depuración	16
15.1. FIX S1 — Retardo por software insuficiente	16
15.2. FIX S2 — Diagnóstico de la función de lectura	16
15.3. FIX S3 — Inicialización tardía del Stack Pointer	16
15.4. Problema Abierto — Error sistemático con operandos multidígito . . .	17
16.Resultados de la Sumadora	17
Proyecto 3 — FemtoUN Boot (Bootloader con SPI SD y PSRAM)	17
17.Contexto y Motivación	18
18.Arquitectura Expandida	18
18.1. Diagrama de Bloques	18
18.2. Cambio Crítico: ADDR_WIDTH = 32	19
19.Mapa de Memoria Expandido	19
20.Periférico SPI	20
20.1. Descripción	20
20.2. Registros del Controlador SPI	20
20.3. Motor SPI (<code>spi_master.v</code>)	20
21.Adaptador de Bus PSRAM (<code>psram_bus.v</code>)	21
21.1. Función	21
21.2. Protocolo de Acceso	21
21.3. Señales del IP PSRAM de Gowin	22
21.4. PLL: Generación de 108 MHz	22

22.Firmware del Bootloader	23
22.1. Flujo de Arranque	23
22.2. Inicialización de la Tarjeta SD	25
22.3. Lectura de Bloques y Copia a PSRAM	25
22.4. Salto a PSRAM	25
22.5. Archivo de Arranque del Bootloader (boot.S)	26
22.6. Linker Script (boot_link.ld)	26
23.Generación de IP Cores en Gowin EDA	26
23.1. IP rPLL (27 MHz $\rightarrow$ 108 MHz)	27
23.2. IP PSRAM_Memory_Interface_HS	27
23.3. Integración en top_tang_boot.v	27
24.Simulación del Bootloader	28
25.Compilación del Firmware	29
26.Estrategia de Desarrollo Incremental	29
27.Estructura del Proyecto	30
28.Conclusiones Generales	31
A. Resumen Consolidado de Hallazgos	32
B. Comandos de Compilación	32

Plataforma Base — SoC RISC-V con UART

Esta parte describe la arquitectura del SoC Femto_Un y sus componentes de hardware: el procesador FemtoRV32, la memoria BRAM, el periférico UART y la infraestructura de comunicación serial. El diseño aquí documentado constituye la **plataforma base** sobre la cual se construyen los tres proyectos posteriores (eco ASCII, sumadora y bootloader), que reutilizan íntegramente estos módulos.

1 Introducción

1.1 Descripción del Proyecto

El proyecto **Femto_Un** consiste en la implementación de un sistema en chip (*System on Chip*, SoC) basado en el procesador softcore de arquitectura RISC-V denominado FemtoRV32, variante *Quark*. Dicho sistema se sintetiza e implementa sobre una FPGA *Tang Primer 20K* de Sipeed, equipada con el chip Gowin GW2A-LV18PG256C8/I7. El SoC integra un procesador RISC-V con soporte para el conjunto de instrucciones RV32I, una memoria de programa y datos tipo *Block RAM* (BRAM) de 32 KiB inicializada con firmware en formato hexadecimal, y un periférico de comunicación serial UART configurable a 120 000 baudios.

El desarrollo se organiza en tres etapas progresivas: la primera establece la arquitectura del SoC y la verifica mediante simulación funcional con Icarus Verilog y GTKWave; la segunda valida el canal de comunicación UART en hardware real mediante un programa de eco ASCII; la tercera construye una aplicación de calculadora básica (sumadora) sobre el canal verificado.

1.2 Objetivos

1.2.1 Objetivo General

Diseñar, implementar, verificar y validar un sistema en chip basado en el procesador softcore RISC-V FemtoRV32 sobre la FPGA Tang Primer 20K, integrando un periférico UART para comunicación serial, y demostrar su funcionalidad mediante aplicaciones embebidas progresivamente más complejas.

1.2.2 Objetivos Específicos

1. Implementar el núcleo FemtoRV32 Quark con soporte completo para el ISA base RV32I.
2. Integrar una memoria BRAM de 32 KiB con escritura granular por bytes.
3. Diseñar un periférico UART mapeado en memoria a 120 000 baudios.
4. Verificar el diseño mediante simulación con Icarus Verilog y GTKWave.
5. Validar el canal UART bidireccional mediante un programa de eco ASCII en hardware.
6. Implementar una calculadora básica que demuestre ejecución de lógica de aplicación.
7. Documentar los hallazgos de depuración como guía para desarrollo futuro.

1.3 Justificación

La arquitectura RISC-V constituye un estándar abierto y libre de regalías que permite a investigadores y estudiantes estudiar, modificar y distribuir implementaciones sin restricciones. Su modularidad (conjunto base RV32I con 47 instrucciones, extensible con M, C, F, etc.), la regularidad de sus formatos de instrucción y la disponibilidad de herramientas (GCC, LLVM, simuladores) la convierten en una plataforma pedagógica de primer orden. La FPGA Tang Primer 20K, con 20 736 LUTs, 15 552 registros y 41 472 bits de BRAM, ofrece recursos suficientes para un SoC completo a costo accesible, permitiendo experimentar con el *hardware-software co-design* de forma integral.

2 Arquitectura General del SoC

2.1 Diagrama de Bloques

El sistema Femto_Un se organiza como un SoC con arquitectura de bus compartido. Un único maestro de bus (la CPU FemtoRV32) accede a múltiples esclavos (BRAM y periférico UART) mediante decodificación de direcciones. La Figura 1 presenta el diagrama de bloques del sistema.

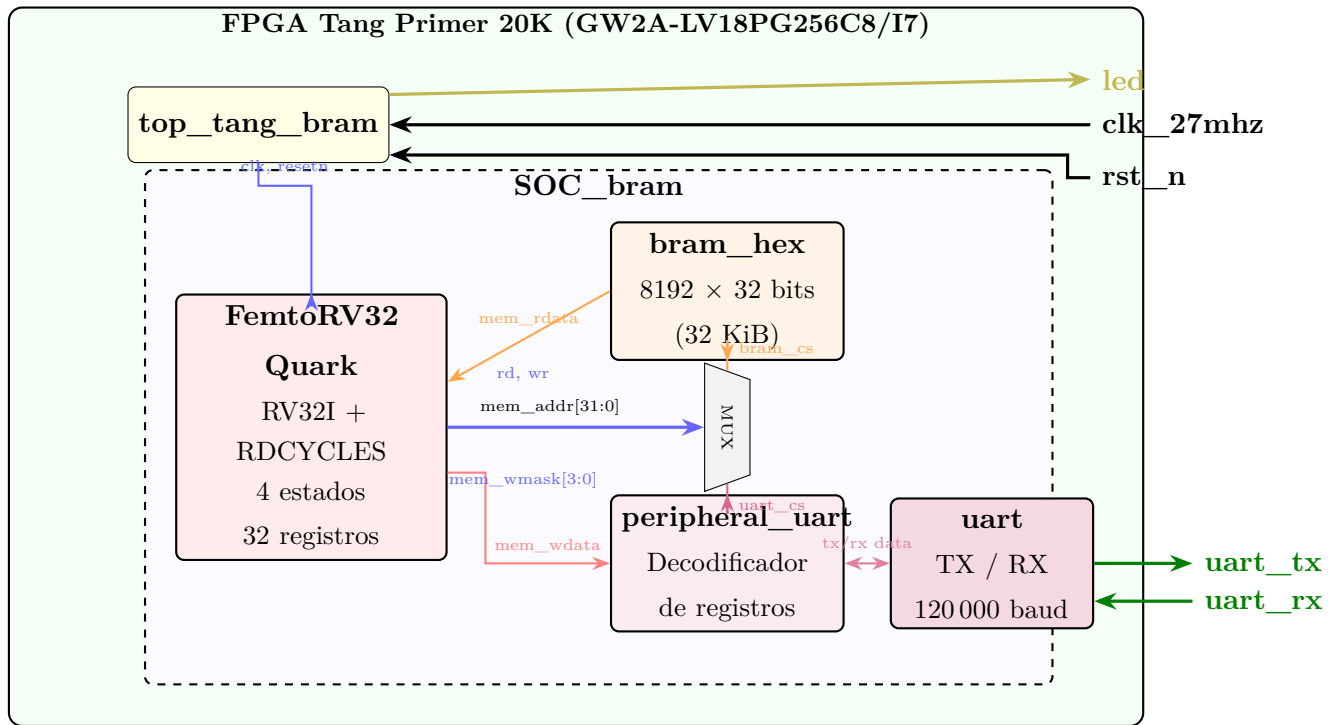


Figura 1: Diagrama de bloques del SoC Femto_Un. La CPU genera direcciones hacia el multiplexor de decodificación, que activa **bram_cs** o **uart_cs** según el rango. Los datos de lectura retornan por **mem_rdata** y los de escritura viajan por **mem_wdata**.

3 Flujo de Datos — Máquina de Estados

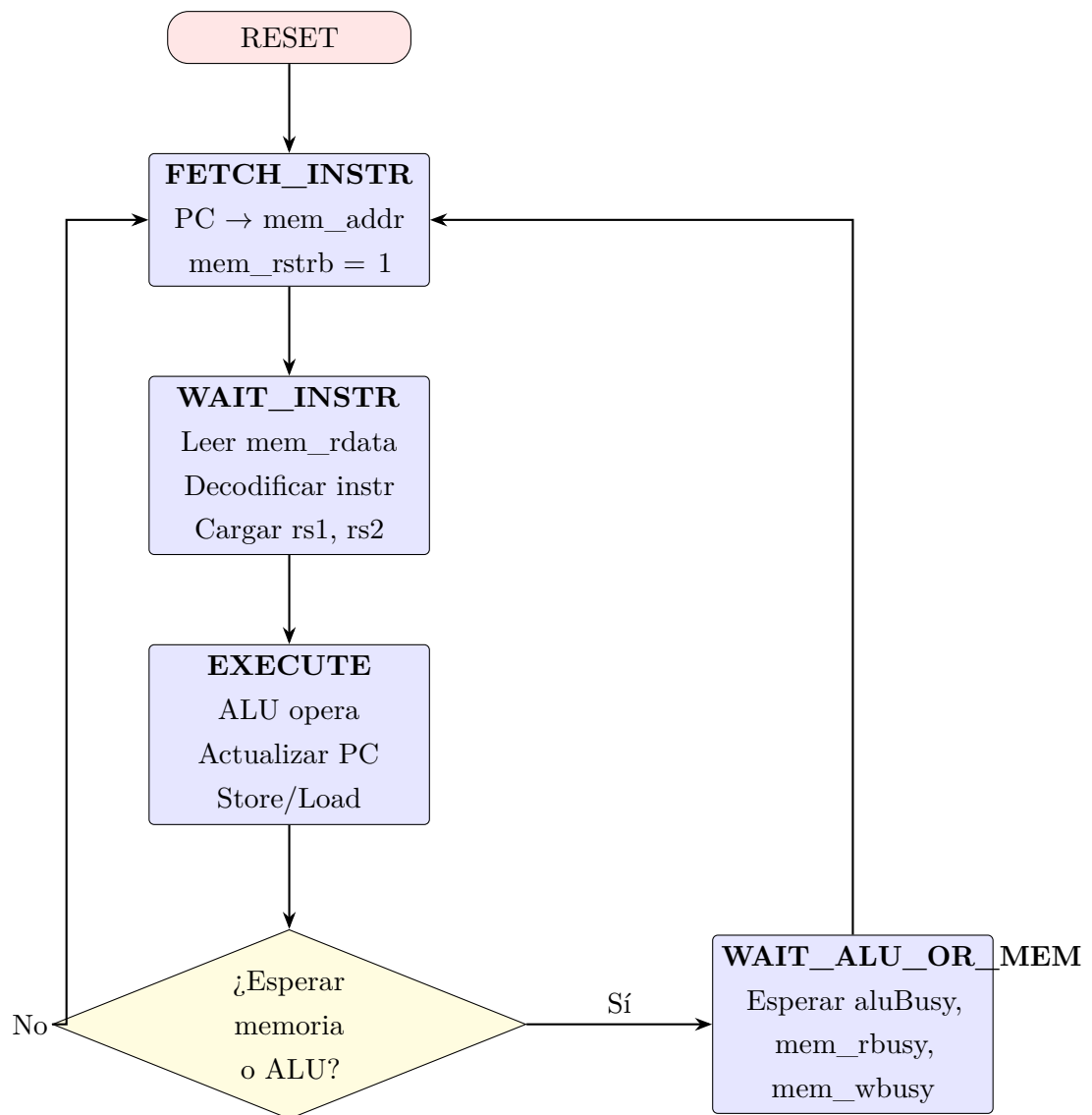


Figura 2: Máquina de estados del procesador FemtoRV32.

3.1 Bus de Memoria

La comunicación entre la CPU y los periféricos utiliza las siguientes señales:

Tabla 1: Señales del bus de memoria del SoC.

Señal	Ancho	Descripción
mem_addr	32 bits	Dirección generada por la CPU.
mem_wdata	32 bits	Datos de escritura (CPU → periférico).
mem_rdata	32 bits	Datos de lectura (periférico → CPU).
mem_wmask	4 bits	Máscara de escritura por bytes.
mem_rstrb	1 bit	Strobe de lectura activa.

La decodificación de direcciones compara `mem_addr[31:16]`: si es `16'h0040` se activa `uart_cs`; en caso contrario, `bram_cs`. Esto crea dos regiones: BRAM en `0x00000000–0x00007FFF` y UART en `0x00400000–0x0040FFFF`.

```

1 wire uart_cs = (mem_addr[31:16] == 16'h0040);
2 wire bram_cs = ~uart_cs;
3 assign mem_rdata = uart_cs ? uart_rdata : bram_rdata;

```

Listing 1: Decodificación de direcciones y multiplexor de lectura.

4 Descripción Detallada de Hardware

4.1 Módulo `top_tang_bram` — Nivel Superior

Actúa como interfaz entre los pines físicos de la FPGA y el SoC. Su responsabilidad principal es generar un reset estable mediante un contador de 20 bits. Tras $2^{20} = 1\,048\,576$ ciclos (≈ 38.8 ms a 27 MHz), la señal `resetn` se activa y la CPU comienza a ejecutar. El LED refleja el estado de `resetn`: apagado durante el reset, encendido cuando el sistema está operativo.

```

1 reg [19:0] reset_cnt = 0;
2 wire resetn = &reset_cnt; // AND-reduction: 1 cuando todos los
  bits son 1
3 always @(posedge clk_27mhz) begin
4     if (!rst_n) reset_cnt <= 0; // Reset externo:
  reiniciar
5     else if (!resetn) reset_cnt <= reset_cnt + 1; // Incrementar
6 end
7 assign led = resetn;

```

Listing 2: Generador de reset del nivel superior.

4.2 Módulo SOC_bram — Integración del Sistema

Conecta el procesador FemtoRV32 con la BRAM y el periférico UART. Define las señales derivadas del bus ($wr = !mem_wmask$, $rd = mem_rstrb$) y propaga el reloj, el reset y las líneas seriales. Las señales mem_rbusy y mem_wbusy se fijan en 1'b0 porque tanto la BRAM como el UART responden en un solo ciclo de reloj.

4.3 Módulo bram_hex — Memoria de Programa y Datos

Memoria síncrona de 8192 palabras de 32 bits (32 KiB), inicializable desde archivo hexadecimal con `$readmemh`. Soporta escritura granular por bytes mediante la máscara $mem_wmask[3:0]$, habilitando las instrucciones SB (store byte), SH (store halfword) y SW (store word) del ISA RISC-V. La dirección de byte se convierte a dirección de palabra descartando los dos bits inferiores: $word_addr = mem_addr[31:2]$.

```

1  reg [31:0] MEM [0:8191];
2  wire [29:0] word_addr = mem_addr[31:2];
3
4  initial $readmemh("src/firmware.hex", MEM);
5
6  always @(posedge clk) begin
7      if (cs & rd) mem_rdata <= MEM[word_addr[12:0]];
8      if (cs & wr) begin
9          if (mem_wmask[0]) MEM[word_addr[12:0]][ 7: 0] <= mem_wdata
10             [ 7: 0];
11          if (mem_wmask[1]) MEM[word_addr[12:0]][15: 8] <= mem_wdata
12             [15: 8];
13          if (mem_wmask[2]) MEM[word_addr[12:0]][23:16] <= mem_wdata
14             [23:16];
15          if (mem_wmask[3]) MEM[word_addr[12:0]][31:24] <= mem_wdata
16             [31:24];
17      end
18  end

```

Listing 3: Lógica de lectura/escritura de la BRAM con máscara por bytes.

5 Módulo FemtoRV32 — Procesador RISC-V

Implementación minimalista del ISA RV32I (Bruno Levy y Matthias Koch). Soporta las 47 instrucciones base más RDCYCLES. Opera con máquina de estados *one-hot* de 4 fases (3–5 ciclos por instrucción). La ALU realiza operaciones combinacionales (suma, resta, comparaciones, lógica) y secuenciales (desplazamientos bit a bit). El archivo de registros (32×32 bits) se escribe en el flanco negativo del reloj para evitar conflictos.

Tabla 2: Instrucciones RV32I decodificadas por FemtoRV32.

Señal	Opcode[6:2]	Tipo	Operación
isLoad	00000	I	$rd \leftarrow \text{mem}[rs1+Imm]$
isALUimm	00100	I	$rd \leftarrow rs1 \text{ OP } Imm$
isStore	01000	S	$\text{mem}[rs1+Simm] \leftarrow rs2$
isALUreg	01100	R	$rd \leftarrow rs1 \text{ OP } rs2$
isLUI	01101	U	$rd \leftarrow Imm$
isAUIPC	00101	U	$rd \leftarrow PC + Imm$
isBranch	11000	B	$\text{if}(\text{cond}) \text{ PC} \leftarrow PC+Bimm$
isJALR	11001	I	$rd \leftarrow PC+4; \text{ PC} \leftarrow rs1+Imm$
isJAL	11011	J	$rd \leftarrow PC+4; \text{ PC} \leftarrow PC+Jimm$
isSYSTEM	11100	I	$rd \leftarrow \text{cycles}$

6 Mapa de Memoria

Tabla 3: Mapa de memoria del SoC Femto_Un.

Inicio	Fin	Tamaño	Periférico	Descripción
0x00000000	0x00007FFF	32 KB	BRAM	Memoria de programa y datos.
0x00400008	0x0040000B	4 B	UART_DATA	Registro de datos TX/RX.
0x00400010	0x00400013	4 B	UART_CTRL	Registro de control y estado.

La secuencia para transmitir un byte consiste en: (1) verificar `tx_busy=0`, (2) escribir el byte en `UART_DATA`, (3) escribir 1 en `UART_CTRL` para activar `tx_wr`, (4) escribir 0 para desactivarlo. La recepción requiere: (1) esperar `rx_avail=1`, (2) leer `UART_DATA`, (3) pulsar `rx_ack`, (4) esperar `rx_avail=0`.

7 Comunicación UART

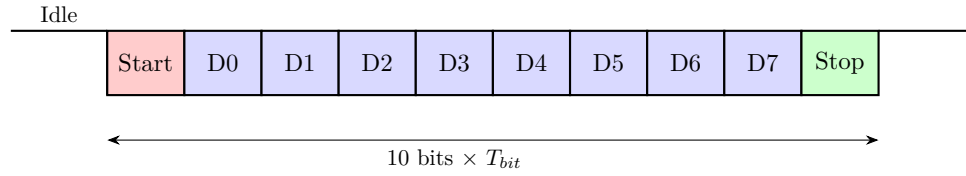


Figura 3: Estructura de trama UART: 1 start, 8 datos (LSB primero), 1 stop.

Tabla 4: Parámetros del sistema.

Parámetro	Valor
FPGA	Tang Primer 20K — GW2A-LV18PG256C8/I7
Frecuencia de reloj	27 MHz
Baud rate UART	120 000 bps (real: 120 535.7, error 0.45 %)
Configuración serial	8N1
Duración de trama	$\approx 83.3 \mu\text{s}$ (2250 ciclos)
Memoria BRAM	8192 palabras de 32 bits (32 KiB)
Compilador	xpack-riscv-none-elf-gcc 15.2.0
Flags	-march=rv32i -mabi=ilp32 -Os -nostartfiles
Terminal	PuTTY — 120000 baud, ISO-8859-1

8 Implementación en FPGA

Tabla 5: Asignación de pines del SoC Femto_Un.

Señal	Pin	I/O	Pull	Drive
clk_27mhz	H11	LVC MOS33	Ninguno	—
rst_n	E8	LVC MOS33	Pull-Up	—
uart_tx	M11	LVC MOS33	Ninguno	8 mA
uart_rx	T13	LVC MOS33	Pull-Up	—
led	L16	LVC MOS33	Ninguno	8 mA

El flujo de implementación sigue las etapas: creación del proyecto en Gowin IDE, síntesis RTL (donde se infieren los bloques BRAM), Place & Route con restricciones de pines, generación del bitstream (.fs) y programación mediante el programador USB integrado.

Proyecto 1 — Verificación del Canal UART (Eco ASCII)

Con la plataforma base descrita en la Parte verificada en simulación, este proyecto valida el canal de comunicación UART en hardware real. El firmware, el mapa de memoria y los módulos Verilog son exactamente los documentados en las secciones anteriores; aquí se añade únicamente el programa en C que ejerce el periférico.

9 Contexto y Objetivo

Confirmado el funcionamiento del SoC en simulación, el siguiente paso crítico es validar la cadena completa de comunicación en hardware real. El programa de eco ASCII recibe cada byte desde la terminal PuTTY y lo devuelve acompañado de su código ASCII decimal (Tecla: X ASCII: N). Su sencillez es deliberada: cualquier comportamiento inesperado se atribuye directamente al canal de comunicación, no a la lógica de aplicación.

10 Diseño del Firmware de Eco

Los registros del periférico se acceden mediante macros `#define` (nunca variables globales — véase FIX E2). La transmisión requiere verificar `tx_busy`, escribir el dato y generar un pulso en `tx_wr`. La recepción requiere esperar `rx_avail`, leer el dato, pulsar `rx_ack` y esperar que `rx_avail` baje (FIX E3). La conversión a decimal usa restas sucesivas porque RV32I no tiene división por hardware.

```

1 void enviar_byte(char c) {
2     while (*uart_ctrl & (1 << 9)); // esperar tx_busy = 0
3     *uart_data = (int)c;
4     *uart_ctrl = 1;                // tx_wr = 1
5     *uart_ctrl = 0;                // tx_wr = 0
6 }
7
8 char recibir_byte(void) {
9     while (!(*uart_ctrl & (1 << 8))); // esperar rx_avail = 1
10    char t = (char)(*uart_data & 0xFF);
11    *uart_ctrl = 2;                // rx_ack = 1
12    *uart_ctrl = 0;                // rx_ack = 0
13    while (*uart_ctrl & (1 << 8)); // esperar rx_avail = 0

```

```
14     return t;  
15 }
```

Listing 4: Funciones de transmisión y recepción del eco ASCII.

11 Hallazgos Durante la Depuración

11.1 FIX E1 — Codificación UTF-8 en la terminal

Síntoma: Caracteres fuera del rango ASCII generaban secuencias multibyte. **Causa:** PuTTY en UTF-8 codifica caracteres no-ASCII como secuencias de 2–4 bytes; el UART los trata como bytes independientes. **Corrección:** Configurar PuTTY con codificación ISO-8859-1 (Latin-1), donde cada carácter ocupa exactamente un byte.

11.2 FIX E2 — Punteros UART como variables globales

Síntoma: Sin salida en terminal; accesos a BRAM en lugar de 0x0040xxxx. **Causa:** Con `-nostartfiles` la sección `.data` nunca se copia a RAM; las variables globales inicializadas contienen basura. **Corrección:** Usar `#define uart_data ((volatile int *)0x00400008)`, que embebe las direcciones como constantes en las instrucciones sin generar variables en `.data`.

11.3 FIX E3 — Doble procesamiento del mismo byte

Síntoma: Cada tecla se imprimía dos veces. **Causa:** `rx_avail` no baja en el mismo ciclo que `rx_ack`; si el firmware evalúa la bandera antes de que expire la latencia, la encuentra activa y procesa el mismo byte otra vez. **Corrección:** Agregar `while (*uart_ctrl & (1 << 8));` tras el pulso.

11.4 FIX E4 — BRAM insuficiente

Síntoma: Síntesis exitosa pero sistema sin respuesta. **Causa:** Firmware más grande que los 256 palabras originales; el sintetizador trunca silenciosamente. **Corrección:** Ampliar a 8192 palabras (32 KiB).

12 Resultados del Eco ASCII

Tabla 6: Verificación de correspondencia carácter-código ASCII en hardware.

Car.	Esp.	Rec.	Cat.	Car.	Esp.	Rec.	Cat.
9	57	57	Dígito	*	42	42	Operador
+	43	43	Operador	c	99	99	Alfa
6	54	54	Dígito	r	114	114	Alfa
=	61	61	Símbolo	CR	13	13	Control
-	45	45	Operador	p	112	112	Alfa

La correspondencia exacta en todos los casos confirma la integridad bidireccional del canal UART. El subsistema queda validado para construir aplicaciones más complejas.

Proyecto 2 — Calculadora Básica (Sumadora)

Este proyecto reutiliza la misma plataforma base (SoC, BRAM de 32 KiB, periférico UART a 120 000 baudios) y construye sobre el canal UART validado en el Proyecto 1. El único cambio respecto al hardware es el firmware cargado en la BRAM.

13 Contexto

Con el canal UART validado, esta etapa construye lógica de aplicación: una calculadora que solicita dos operandos y un operador por terminal, ejecuta la operación y devuelve el resultado. Este proyecto cubre la operación de suma.

14 Diseño del Firmware

14.1 Acumulación de Dígitos sin Multiplicación

El ISA base RV32I no incluye multiplicación por hardware. El operador `*` genera llamadas a `__mulsi3`, no disponible con `-nostartfiles`. La acumulación se implementa con desplazamientos: `num = (num << 3) + (num << 1) + digito`, algebraicamente equivalente a `num = num*10 + digito`.

14.2 Impresión sin División

Análogamente, `/` y `%` generan llamadas a `__divsi3`/`__modsi3`. La conversión a decimal usa restas sucesivas por potencia de diez, cubriendo hasta 10^9 .

14.3 Lazo Principal

El firmware implementa tres fases secuenciales: (1) captura del primer operando dígito a dígito hasta Enter, (2) espera del operador `+`, (3) captura del segundo operando. Tras completarlas, calcula la suma, imprime el resultado y reinicia el lazo.

```
1 while (1) {
2     int num1 = 0, num2 = 0;
3     imprimir_str("\r\n===_SUMADORA_BASICA_===\r\n");
4     imprimir_str("Numero_1_y_Enter:_");
```



```

5     while (1) {
6         char t = leer_char();
7         if (t >= '0' && t <= '9') {
8             num1 = (num1 << 3) + (num1 << 1) + (t - '0');
9             enviar_char(t); // eco del dígito
10        } else if (t == '\r' || t == '\n') break;
11    }
12    /* ... captura operador y num2 ... */
13    imprimir_str("\r\nResultado:");
14    imprimir_numero(num1 + num2);
15 }

```

Listing 5: Lazo principal de la sumadora (fragmento).

15 Hallazgos Durante la Depuración

15.1 FIX S1 — Retardo por software insuficiente

Síntoma: Con dos dígitos, el carácter se duplicaba. **Causa:** Un `for` de 50 000 iteraciones como retardo era reducido por `-Os`; el firmware procesaba el byte antes de que `rx_avail` bajara. **Corrección:** Espera activa sobre la bandera, como en el eco ASCII.

15.2 FIX S2 — Diagnóstico de la función de lectura

Se probaron tres variantes de `leer_char` con un programa de diagnóstico. Las tres produjeron exactamente un byte por tecla con entrada controlada, confirmando que la lectura individual era correcta y el problema residía en el estado del stack.

15.3 FIX S3 — Inicialización tardía del Stack Pointer

Síntoma: Resultados incorrectos con operandos de dos dígitos (error proporcional). **Causa:** `asm volatile("li sp, ...")` dentro de `main` ejecutaba después del prólogo del compilador, que usaba el stack con SP no inicializado. **Corrección:** Archivo `start.S` que inicializa SP antes de saltar a `main`:

```

1 .section .text
2 .global _inicio
3 _inicio:
4     li sp, 0x00007FF0    # stack al final de la BRAM
5     j main               # transferir control
6 loop:
7     j loop               # atrapar retorno inesperado

```

Listing 6: Archivo `start.S` — inicialización previa a `main`.

15.4 Problema Abierto — Error sistemático con operandos multidígito

Tras las correcciones, operandos de un dígito producen resultados correctos; con dos o más dígitos se observa un error de -7 por dígito adicional (e.g., $10 + 1 = 4$ en lugar de 11). La hipótesis principal es que `_inicio` queda ubicado fuera de `0x00000000` al compilar con `-nostdlib -lgcc`, causando ejecución incorrecta al arranque. La solución recomendada es un linker script explícito.

16 Resultados de la Sumadora

Tabla 7: Resultados experimentales de la sumadora.

Num1	Num2	Esperado	Obtenido	Estado
2	2	4	4	Correcto
9	1	10	10	Correcto
4	6	10	10	Correcto
10	1	11	4	Error (-7)
10	2	12	5	Error (-7)
20	1	21	7	Error (-14)

Proyecto 3 — FemtoUN Boot (Bootloader con SPI SD y PSRAM)

Este proyecto extiende la plataforma base con dos periféricos nuevos (SPI y adaptador PSRAM) y reemplaza el firmware de BRAM por un bootloader. El procesador FemtoRV32, el núcleo UART y la estructura de bus documentados en la Parte se con-

servan sin modificaciones; los cambios se limitan al integrador `SOC_boot.v`, el módulo top-level y el firmware.

17 Contexto y Motivación

Las tres etapas anteriores del proyecto demostraron el funcionamiento del SoC Femto_Un con un programa almacenado exclusivamente en BRAM interna. Sin embargo, la capacidad de 32 KiB de BRAM impone una limitación severa para aplicaciones de mayor complejidad. Esta cuarta etapa expande el sistema con dos periféricos adicionales: un controlador SPI para lectura de tarjetas MicroSD y un adaptador de bus para la memoria PSRAM HyperRAM integrada en la Tang Primer 20K (8 MB). El resultado es un **bootloader** que arranca desde la BRAM, lee un programa de usuario desde la tarjeta SD vía SPI, lo copia a la PSRAM y transfiere el control de ejecución a esa memoria externa, multiplicando por 256 la capacidad de programa disponible.

18 Arquitectura Expandida

18.1 Diagrama de Bloques

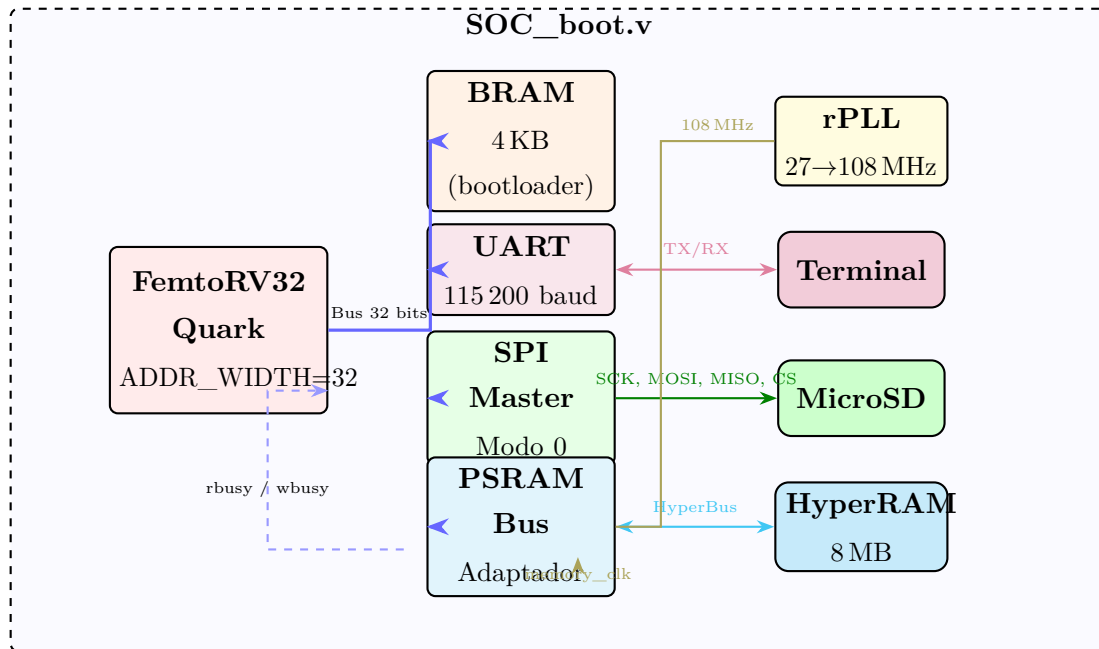


Figura 4: Arquitectura expandida del SoC FemtoUN Boot. Se añaden el controlador SPI para lectura de MicroSD, el adaptador de bus PSRAM para acceso a HyperRAM de 8 MB, y un PLL que genera los 108 MHz requeridos por la PSRAM.

18.2 Cambio Crítico: ADDR_WIDTH = 32

El FemtoRV32 original utiliza ADDR_WIDTH = 24, lo que limita el espacio de direcciones a 16 MB (0x000000–0xFFFFF). Para acceder a la PSRAM en 0x20000000 se requiere ampliar a 32 bits:

```
1 FemtoRV32 #(
2     .RESET_ADDR (32'h00000000),
3     .ADDR_WIDTH (32)
4 ) CPU ( ... );
```

Listing 7: Instanciación del procesador con bus de 32 bits completo.

Esto incrementa ligeramente el uso de LUTs pero es necesario para el mapa de memoria expandido.

19 Mapa de Memoria Expandido

Tabla 8: Mapa de memoria del SoC FemtoUN Boot.

Dirección	Periférico	Tamaño	Descripción
0x00000000	BRAM	4 KB	Bootloader (código de arranque, solo lectura en runtime).
0x00400008	UART_DATA	4 B	Registro de datos TX/RX.
0x00400010	UART_CTRL	4 B	Control y estado del UART.
0x00500000	SPI_DATA	4 B	Byte a transmitir / último byte recibido [7:0].
0x00500004	SPI_CTRL	4 B	Chip select y divisor de reloj.
0x00500008	SPI_STATUS	4 B	Estado del controlador SPI (solo lectura).
0x20000000	PSRAM	8 MB	Memoria externa HyperRAM (programa de usuario).

La decodificación de direcciones en SOC_boot.v se amplía para cubrir los cuatro periféricos, comparando campos de bits de la dirección:

```
1 wire uart_cs  = (mem_addr[31:16] == 16'h0040); // 0x0040xxxx
2 wire spi_cs   = (mem_addr[31:16] == 16'h0050); // 0x0050xxxx
3 wire psram_cs = (mem_addr[31:29] == 3'b001);   // 0x20000000+
4 wire bram_cs  = ~uart_cs & ~spi_cs & ~psram_cs; // resto -> BRAM
```

Listing 8: Decodificación de direcciones para cuatro periféricos.

20 Periférico SPI

20.1 Descripción

El subsistema SPI se compone de dos módulos: `spi_master.v`, que implementa el motor SPI en modo 0 (CPOL=0, CPHA=0) con reloj configurable, y `perip_spi.v`, que actúa como interfaz entre el bus de memoria del SoC y el motor SPI, exponiendo tres registros mapeados en memoria.

20.2 Registros del Controlador SPI

Tabla 9: Registros del periférico SPI.

Dirección	Nombre	Acceso	Descripción
0x00500000	SPI_DATA	R/W	Escritura: byte a transmitir [7:0]; inicia la transferencia automáticamente. Lectura: último byte recibido [7:0] (dato capturado en MISO).
0x00500004	SPI_CTRL	R/W	Bit [0]: CS_n — 0 = chip select activo (seleccionar SD), 1 = inactivo. Bits [15:8]: clk_div — divisor de reloj SPI.
0x00500008	SPI_STATUS	R	Bit [0]: busy — 1 = transferencia SPI en progreso.

El reloj SPI se deriva del reloj del sistema mediante el divisor configurable:

$$f_{SCK} = \frac{f_{clk}}{2 \times (\text{clk_div} + 1)} \quad (1)$$

Para la inicialización de la tarjeta SD se requiere $f_{SCK} \leq 400 \text{ kHz}$, lo que se logra con `clk_div = 33`: $f_{SCK} = 27\,000\,000 / (2 \times 34) \approx 397 \text{ kHz}$. Una vez inicializada, la velocidad se incrementa a 6.75 MHz con `clk_div = 1`.

20.3 Motor SPI (`spi_master.v`)

El motor implementa el protocolo SPI en modo 0: el dato se captura en el flanco positivo de SCK y se actualiza en el flanco negativo. Cada transferencia procesa 8 bits (un byte) simultáneamente en ambas direcciones (full-duplex), desplazando bit a bit desde MSB. La señal `busy` se activa al escribir en `SPI_DATA` y se desactiva tras completar los 8 ciclos de reloj SPI.

21 Adaptador de Bus PSRAM (`psram_bus.v`)

21.1 Función

El módulo `psram_bus.v` traduce las transacciones del bus de memoria del FemtoRV32 (señales `mem_addr`, `mem_wdata`, `mem_wmask`, `mem_rstrb`) al protocolo del IP `PSRAM_Memory_Interface_HS` de Gowin, que controla los chips HyperRAM físicos. Este módulo es necesario porque el IP de Gowin utiliza un protocolo de handshake con señales `cmd`, `cmd_en`, `rd_data_valid` que no coinciden directamente con el bus del FemtoRV32.

21.2 Protocolo de Acceso

Las escrituras a PSRAM se ejecutan en un ciclo: el adaptador genera un pulso `cmd_en` con `cmd = 1` (escritura). Las lecturas requieren esperar la señal `rd_data_valid` del IP, durante lo cual el adaptador mantiene `mem_rbusy = 1` para detener la CPU. Este mecanismo aprovecha la capacidad nativa del FemtoRV32 de entrar en el estado `WAIT_ALU_OR_MEM` cuando `mem_rbusy` o `mem_wbusy` están activos.

21.3 Señales del IP PSRAM de Gowin

Tabla 10: Puertos principales del IP PSRAM_Memory_Interface_HS.

Puerto	Dir.	Ancho	Descripción
clk	In	1 bit	Reloj del sistema (27 MHz).
memory_clk	In	1 bit	Reloj de memoria (108 MHz, de la PLL).
pll_lock	In	1 bit	Señal de PLL estabilizada.
rst_n	In	1 bit	Reset activo bajo.
cmd	In	1 bit	0 = lectura, 1 = escritura.
cmd_en	In	1 bit	Pulso para ejecutar el comando.
addr	In	23 bits	Dirección byte ($2^{23} = 8$ MB).
wr_data	In	32 bits	Datos a escribir.
data_mask	In	4 bits	Máscara de bytes (0 = escribir).
rd_data	Out	32 bits	Datos leídos.
rd_data_valid	Out	1 bit	Dato de lectura válido.
init_calib	Out	1 bit	Calibración completa (1 = listo).
o_psram_ck	Out	2 bits	Reloj hacia chips HyperRAM.
IO_psram_dq	I/O	16 bits	Bus de datos bidireccional.
IO_psram_rwds	I/O	2 bits	Señal de strobe.
o_psram_cs_n	Out	2 bits	Chip select de los dos chips.

21.4 PLL: Generación de 108 MHz

La PSRAM requiere un reloj de memoria de al menos $4\times$ el reloj del sistema. El IP rPLL de Gowin multiplica los 27 MHz de entrada:

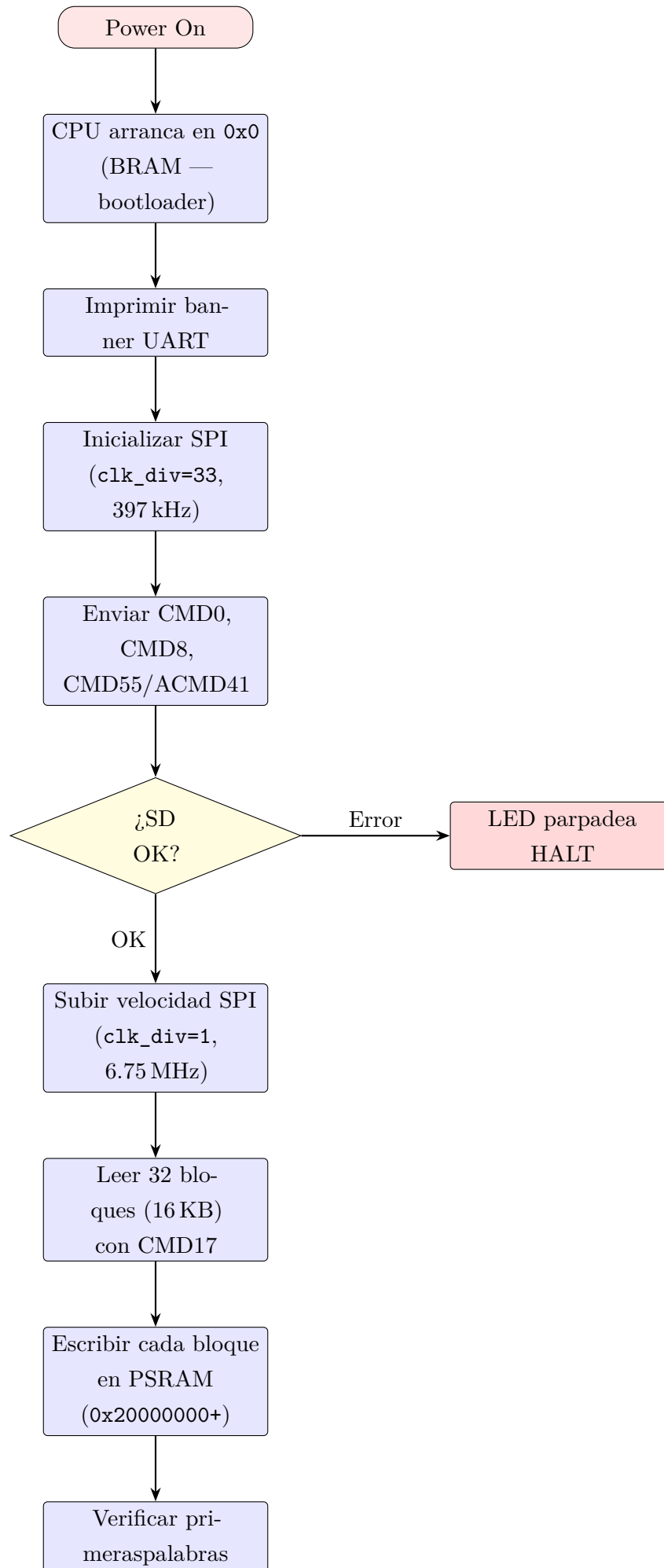
$$f_{mem} = 27 \text{ MHz} \times \frac{\text{FBDIV_SEL} + 1}{\text{IDIV_SEL} + 1} = 27 \times \frac{4}{1} = 108 \text{ MHz} \quad (2)$$

La señal pll_lock indica que la PLL se ha estabilizado y el sistema puede comenzar a operar.

22 Firmware del Bootloader

22.1 Flujo de Arranque

El bootloader reside en la BRAM (4 KB) y se ejecuta al encender la FPGA. Su función es inicializar la tarjeta MicroSD mediante el protocolo SPI, leer un programa de usuario almacenado en los primeros sectores de la tarjeta, copiarlo a la PSRAM y transferir la ejecución a la dirección 0x20000000.

**Figura 5:** Flujo de arranque del bootloader FemtoUN Boot.

22.2 Inicialización de la Tarjeta SD

La inicialización de una tarjeta SD mediante SPI sigue una secuencia estandarizada. Primero se envían al menos 74 pulsos de reloj con CS inactivo para despertar la tarjeta. Luego se ejecutan tres comandos:

Tabla 11: Secuencia de comandos para inicialización de la tarjeta SD.

Comando	Bytes	Respuesta	Función
CMD0	40 00 00 00 00 95	0x01	Reset; entrar en modo SPI (idle).
CMD8	48 00 00 01 AA 87	0x01 + echo	Verificar voltaje; identificar SD v2.0+.
ACMD41	CMD55 + CMD41	0x00	Inicializar y salir del estado idle. Se repite en bucle hasta que la respuesta sea 0x00.

22.3 Lectura de Bloques y Copia a PSRAM

Una vez inicializada la SD, el bootloader lee 32 bloques de 512 bytes (16 KB total) mediante el comando CMD17 (lectura de bloque individual). Cada bloque se lee byte a byte vía SPI, se ensambla en palabras de 32 bits en orden *little-endian* y se escribe en la PSRAM a partir de la dirección 0x20000000:

```

1 void sd_read_block(uint32_t sector, uint32_t *dest) {
2     sd_send_cmd(CMD17, sector);           // solicitar bloque
3     sd_wait_data_token();                 // esperar token 0xFE
4     for (int i = 0; i < 128; i++) { // 512 bytes = 128 words
5         uint32_t word = 0;
6         word |= spi_transfer(0xFF);       // byte 0 (LSB)
7         word |= spi_transfer(0xFF) << 8;  // byte 1
8         word |= spi_transfer(0xFF) << 16; // byte 2
9         word |= spi_transfer(0xFF) << 24; // byte 3 (MSB)
10        dest[i] = word;                   // escribir a PSRAM
11    }
12    spi_transfer(0xFF); spi_transfer(0xFF); // descartar CRC
13 }
```

Listing 9: Pseudocódigo de lectura de un bloque y escritura a PSRAM.

22.4 Salto a PSRAM

Tras copiar y verificar el programa de usuario, el bootloader ejecuta un salto incondicional a la dirección base de la PSRAM. Esto se implementa mediante un

puntero a función en C:

```
1 void (*entry)(void) = (void (*)(void))0x20000000;
2 entry(); // equivale a: jalr x0, x_reg, 0 (salto sin retorno)
```

Listing 10: Transferencia de control a la PSRAM.

El programa de usuario debe contener su propio código de arranque (`user_start.S`) que configure el puntero de pila dentro del espacio de la PSRAM antes de llamar a `main`.

22.5 Archivo de Arranque del Bootloader (`boot.S`)

```
1 .section .text
2 .global _start
3 _start:
4     li sp, 0x00001000    # stack al final de la BRAM (4 KB)
5     j main
6 .loop:
7     j .loop
```

Listing 11: Código de arranque del bootloader.

22.6 Linker Script (`boot_link.ld`)

```
1 ENTRY(_start)
2 MEMORY {
3     BRAM (rwx) : ORIGIN = 0x00000000, LENGTH = 4K
4 }
5 SECTIONS {
6     .text : { *(.text*) } > BRAM
7     .rodata : { *(.rodata*) } > BRAM
8     .data : { *(.data*) } > BRAM
9     .bss : { *(.bss*) } > BRAM
10 }
```

Listing 12: Linker script del bootloader (ubicado en BRAM).

23 Generación de IP Cores en Gowin EDA

La implementación en hardware requiere dos IP cores propietarios de Gowin que no pueden ser sintetizados a partir de código Verilog genérico: la PLL para generación de reloj y el controlador PSRAM HyperRAM.

23.1 IP rPLL (27 MHz → 108 MHz)

1. En Gowin EDA: **Tools** → **IP Core Generator**.
2. Expandir **Hard Module** → **CLOCK** → **rPLL**.
3. Configurar: CLKIN = 27 MHz, CLKOUT = 108 MHz, marcar LOCK output.
4. Gowin calcula automáticamente los parámetros: IDIV_SEL = 0, FBDIV_SEL = 3.
5. El módulo generado (Gowin_rPLL) expone tres puertos: `clk_in` (entrada 27 MHz), `clkout` (salida 108 MHz), `lock` (PLL estabilizada).

23.2 IP PSRAM_Memory_Interface_HS

1. En Gowin EDA: **Tools** → **IP Core Generator**.
2. Expandir **Hard Module** → **MEMORY** → **PSRAM_Memory_Interface_HS**.
3. Configurar:
 - Device: x2 (ambos chips HyperRAM del Tang Primer 20K).
 - Data Width: 32 bits (ancho del bus del FemtoRV32).
 - Address Width: 23 bits ($2^{23} = 8\text{ MB}$).
 - System Clock: 27 MHz; Memory Clock: 108 MHz.
 - Marcar **Enable data mask** (necesario para escrituras por bytes).
4. El IP genera múltiples archivos `.v` que se agregan automáticamente al proyecto.
5. Los pines físicos de HyperRAM (`0_psram_ck`, `IO_psram_dq`, etc.) están conectados al die de forma fija y **no requieren constraints manuales** en el archivo CST.

23.3 Integración en top_tang_boot.v

Tras generar ambos IPs, se descomentean las instanciaciones en el archivo top-level y se eliminan las asignaciones temporales:

```

1 // PLL: 27 MHz -> 108 MHz
2 Gowin_rPLL pll_inst (
3     .clk_in  (clk_27mhz ),
4     .clkout  (memory_clk),
5     .lock    (pll_lock  )

```

```

6 );
7
8 // PSRAM HyperRAM
9 PSRAM_Memory_Interface_HS_Top psram_ip (
10     .clk            (clk_27mhz            ),
11     .memory_clk     (memory_clk           ),
12     .pll_lock       (pll_lock             ),
13     .rst_n          (sys_resetn           ),
14     .O_psram_ck     (O_psram_ck           ),
15     .O_psram_ck_n   (O_psram_ck_n        ),
16     .IO_psram_rwds   (IO_psram_rwds       ),
17     .IO_psram_dq     (IO_psram_dq         ),
18     .O_psram_reset_n(O_psram_reset_n     ),
19     .O_psram_cs_n    (O_psram_cs_n        ),
20     .cmd             (psram_cmd           ),
21     .cmd_en         (psram_cmd_en        ),
22     .addr            (psram_addr          ),
23     .wr_data         (psram_wr_data       ),
24     .data_mask       (psram_data_mask     ),
25     .rd_data         (psram_rd_data       ),
26     .rd_data_valid   (psram_rd_data_valid),
27     .init_calib      (psram_init_calib    )
28 );

```

Listing 13: Instanciación de PLL y PSRAM IP en el top-level.

24 Simulación del Bootloader

La simulación utiliza modelos comportamentales de la tarjeta SD (`sd_model.v`) y la PSRAM (`psram_model.v`) que reproducen el protocolo de comunicación sin requerir los IP propietarios de Gowin. Las señales clave a verificar en GTKWave son:

Tabla 12: Señales clave a verificar en la simulación del bootloader.

Señal(es)	Verificación
<code>sd_sck</code> , <code>sd_mosi</code> , <code>sd_cs_n</code>	Secuencia SPI: CS bajo → 48 bits de comando → respuesta.
<code>psram_cmd_en</code> , <code>psram_addr</code>	Escrituras secuenciales a PSRAM con direcciones incrementales.
<code>mem_addr</code>	Transición de <code>0x0000xxxx</code> a <code>0x2000xxxx</code> tras el salto.
Mensajes UART (stdout)	Banner del bootloader, confirmaciones de CMD0/CMD8/ACMD41, salto.

```
cd fw/ && make all          # compilar bootloader + programa de
                             usuario
cd ../sim/ && make sim      # ejecutar simulacion
make wave                  # abrir GTKWave
```

Listing 14: Compilación y simulación del bootloader.

25 Compilación del Firmware

```
# Bootloader (se almacena en BRAM via $readmemh)
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -Os ^
    -nostartfiles -nostdlib -lgcc -T boot_link.ld ^
    -o bootloader.elf boot.S bootloader.c
riscv64-unknown-elf-objcopy -O verilog --verilog-data-width=4 ^
    bootloader.elf bootloader.hex

# Programa de usuario (se graba en MicroSD, se copia a PSRAM)
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -Os ^
    -nostartfiles -nostdlib -lgcc -T user_link.ld ^
    -o user_test.elf user_start.S user_test.c
riscv64-unknown-elf-objcopy -O binary user_test.elf user_test.bin

# Grabar en MicroSD (sector 0)
sudo dd if=user_test.bin of=/dev/sdX bs=512 seek=0 conv=notrunc
```

Listing 15: Compilación del bootloader y programa de usuario.

26 Estrategia de Desarrollo Incremental

La integración del bootloader se recomienda en tres fases para facilitar la depuración:

Tabla 13: Fases de desarrollo incremental del bootloader.

Fase	Componente	Verificación
1	SPI solo (sin PSRAM)	Firmware de prueba que envía CMD0 y lee respuesta. Verificar con osciloscopio o analizador lógico.
2	PSRAM solo (sin SD)	Firmware que escribe/lee palabras en 0x20000000. Verificar integridad de datos por UART.
3	Bootloader completo	Integrar ambos periféricos. Verificar en simulación con modelos, luego en hardware con SD real.

27 Estructura del Proyecto

```
femtoun_boot/
  rtl/
    SOC_boot.v           SoC integrador (4 perifericos)
    top_tang_boot.v      Top-level (PLL + PSRAM IP)
    spi_master.v         Motor SPI modo 0
    perip_spi.v          Periferico SPI mapeado en memoria
    psram_bus.v          Adaptador bus FemtoRV32 <-> PSRAM IP
    femtorv_quark.v      CPU FemtoRV32 (ADDR_WIDTH=32)
    bram_hex.v           BRAM 4 KB con inicializacion hex
    perip_uart.v         Periferico UART (sin cambios)
    uart.v              Core UART (sin cambios)
  fw/
    boot.S              Startup del bootloader
    bootloader.c        Codigo C del bootloader
    boot_link.ld         Linker script (BRAM)
    user_start.S         Startup del programa de usuario
    user_test.c          Programa de prueba para PSRAM
    user_link.ld         Linker script (PSRAM)
    Makefile
  sim/
    tb_soc_boot.v        Testbench principal
    sd_model.v           Modelo comportamental de tarjeta SD
    psram_model.v        Modelo comportamental de PSRAM
  constraints/
    tang_boot.cst        Constraints de pines
    tang_boot.sdc        Constraints de timing
```

Listing 16: Estructura de archivos del proyecto FemtoUN Boot.

28 Conclusiones Generales

1. **Viabilidad demostrada.** Se implementó un SoC RISC-V completo sobre FPGA de bajo costo, confirmando que las arquitecturas abiertas son plataformas viables para enseñanza e investigación.
2. **Verificación progresiva eficaz.** La metodología en tres etapas (simulación → eco → aplicación) permitió aislar problemas en cada nivel antes de avanzar.
3. **Canal UART íntegro.** La correspondencia exacta entre caracteres y códigos ASCII confirma el correcto funcionamiento bidireccional del subsistema serial.
4. **Gestión de `rx_avail` obligatoria.** La espera activa tras `rx_ack` es requisito no negociable; omitirla produce doble procesamiento silencioso.
5. **Direcciones como constantes de compilación.** Las macros `#define` eliminan la dependencia de `.data`, inexistente con `-nostartfiles`.
6. **Inicialización de SP previa a `main`.** El archivo `start.S` resuelve estructuralmente la dependencia del prólogo del compilador.
7. **Limitaciones del ISA base.** Sin extensión M, la aritmética decimal requiere desplazamientos y restas sucesivas, incrementando tamaño de código y tiempo de ejecución.
8. **Linker script necesario.** El problema abierto señala la necesidad de controlar explícitamente la ubicación de `_inicio` en `0x00000000`.
9. **Nueve hallazgos documentados** (E1–E4 del eco, S1–S4 de la sumadora, más la corrección de simulación) constituyen una guía práctica para firmware bare-metal sobre FemtoRV32.

A Resumen Consolidado de Hallazgos

Tabla 14: Los 9 hallazgos documentados durante el desarrollo de Femto_Un.

ID	Etap	Componente	Problema	Corrección
E1	Eco	PuTTY	UTF-8 multibyte	ISO-8859-1
E2	Eco	main.c	Punteros en .data	#define
E3	Eco	main.c	Doble lectura	Espera rx_avail=0
E4	Eco	bram_hex.v	BRAM 256 palabras	Ampliar a 8192
S1	Suma	leer_char	Retardo con -Os	Espera activa
S2	Suma	leer_char	Diagnóstico	3 variantes: OK
S3	Suma	main.c	SP tardío	start.S
S4	Suma	linkeo	Error -7/dígito	Abierto
–	Sim.	femtorv.v	Registros con X	Init. en BENCH

B Comandos de Compilación

```
riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -Os -nostartfiles ^
-Wl,-Ttext=0x00000000" -o firmware.elf main.c
riscv-none-elf-objcopy -O verilog --verilog-data-width=4 firmware.
elf firmware.hex
```

Listing 17: Eco ASCII.

```
riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -Os -nostartfiles -
nostdlib ^
-lgcc "-Wl,-Ttext=0x00000000" "-Wl,-e_inicio" ^
-o firmware.elf start.S main.c
riscv-none-elf-objcopy -O verilog --verilog-data-width=4 firmware.
elf firmware.hex
```

Listing 18: Sumadora con start.S.

```
iverilog -DBENCH -o sim.out tb/tb_femtoUN.v src/*.v
vvp sim.out
gtkwave femtoUN.vcd femtoUN.gtkw
```

Listing 19: Simulación.

Referencias

- [1] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, V20191213, RISC-V Foundation, 2019.
- [2] B. Levy, “FemtoRV32: A minimalistic RISC-V CPU,” GitHub, 2021. <https://github.com/BrunoLevy/learn-fpga>
- [3] Gowin Semiconductor, *GW2A Series of FPGA Products — Data Sheet*, V1.9, 2024.
- [4] Sipeed, *Tang Primer 20K User Guide*, 2023. <https://wiki.sipeed.com/hardware/en/tang/tang-primer-20k/primer-20k.html>
- [5] J. Axelson, *Serial Port Complete*, 2nd ed., Lakeview Research LLC, 2007.
- [6] D.A. Patterson and J.L. Hennessy, *Computer Organization and Design RISC-V Edition*, 2nd ed., Morgan Kaufmann, 2020.
- [7] S. Harris and D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2021.
- [8] S. Williams, “Icarus Verilog,” <http://iverilog.icarus.com/>
- [9] T. Bybell, “GTKWave Electronic Waveform Viewer,” <http://gtkwave.sourceforge.net/>
- [10] xPack Project, “xPack GNU RISC-V Embedded GCC,” <https://xpack.github.io/dev-tools/riscv-none-elf-gcc/>